

# CoDiM: Contrastive Distillation into Generative Models for Scalable Dataset Distillation

Quang Duy Hoang, Nam Thang Hoang  
*Faculty of Information Technology*  
*Hanoi University of Civil Engineering (HUCE)*  
Hanoi, Vietnam  
duypo03cbvn@gmail.com, thanghn@huce.edu.vn

Huy Hieu Nguyen, Hai Anh Tran  
*School of Information and Communication Technology (SOICT)*  
*Hanoi University of Science and Technology (HUST)*  
Hanoi, Vietnam  
huyhieu1422002@gmail.com, anhhth@soict.hust.edu.vn

Truong X. Tran  
*School of Science, Engineering and Technology*  
*Penn State University*  
Penn State Harrisburg, USA  
truong.tran@psu.edu

**Abstract**—In recent years, the rapid growth of big data and the increasing demand for digital transformation have shifted the paradigm from data scarcity to data abundance. As data are generated at an unprecedented rate, significant challenges arise in terms of storage requirements and computational costs for model training. To address these, Dataset Distillation has emerged as a promising approach that compresses the knowledge of a large-scale dataset into a compact synthetic dataset, thereby reducing storage demands and accelerating training. However, existing methods predominantly rely on direct optimization in pixel space, which suffers from limited scalability and high computational overhead. In this paper, we propose Contrastive Distill into Model (CoDiM), a novel framework that integrates the InfoNCE loss from Contrastive Learning with the structural principles of Generative Adversarial Networks (GANs). Our approach improves sample diversity while minimizing the discrepancy between logits produced by real and synthetic data. Extensive experiments on Fashion-MNIST, CIFAR-10, and SVHN demonstrate the effectiveness of the proposed method. Notably, CoDiM achieves 67.01% accuracy on CIFAR-10 with only 50 IPC, highlighting its efficiency in data-constrained scenarios.

**Index Terms**—Dataset Distillation, Contrastive Learning, Data Reusability, Reusable AI, Computer Vision

## I. INTRODUCTION

Recent breakthroughs in Computer Vision (CV) and Natural Language Processing (NLP) [1], [2] are largely driven by the expansion of data scale and diversity. Beyond architectural advancements, large-scale datasets provide the necessary informational density to capture complex nonlinear features, ensuring high performance and robustness. However, this growth introduces significant resource trade-offs. The resulting computational burden in terms of energy, storage, and processing often exceeds the capacity of conventional systems, hindering deployment in resource constrained or edge environments. Given the diminishing returns of continued dataset scaling [3], there is an urgent need for data efficient learning methodologies.

Dataset Distillation (DD) has emerged as a promising approach to reduce the data and computational requirements of

deep learning models [4]. The goal of DD is to synthesize a compact dataset that can retain a highly representative distribution and essential information of the original dataset. As a result, this approach significantly reduces storage requirements, energy consumption, and training time while maintaining competitive performance compared to training on the full dataset [5]. The training process in DD can be formulated as a bi-level optimization problem that involves alternating updates between model parameter optimization in the inner loop and synthetic data refinement in the outer loop, thereby aligning the synthetic samples with the real data distribution.

To mitigate the computational overhead, many methods align statistical properties between real and synthetic datasets [6]–[10]. Despite notable progress, these approaches still suffer from scalability issues. As the number of images per class (IPC) increases, computational complexity grows rapidly, especially for high-resolution datasets. The resulting heavy gradient computations and memory access lead to slow convergence, making the distillation phase a major time and cost bottleneck despite the efficiency gains in downstream training.

To address these limitations, we propose CoDiM (Contrastive Distill into Model). Unlike conventional methods relying on pixel space optimization, CoDiM shifts the paradigm toward parameterized data representations. By leveraging the Generator network within an adversarial GAN-based framework, our approach captures the underlying source distribution rather than maintaining a static sample set, and as a result, bypasses the redundant and costly re-optimization cycles inherent in pixel-space methods. Once trained, CoDiM enables instantaneous data synthesis for any specified budget via a single forward pass, reducing the overall computational burden. The main contributions of this work are summarized as follows:

- 1) We introduce CoDiM, a GAN-based framework that stabilizes dataset distillation through a dual-loss objective. Specifically, we integrate InfoNCE to enforce intra-class diversity and Logit loss to capture critical training

signals, such as gradients and decision boundaries. This approach ensures high semantic fidelity, leading to superior performance in downstream task efficiency.

- 2) We conduct evaluations across three benchmarks: Fashion-MNIST, CIFAR-10, and SVHN. Empirical results demonstrate that CoDiM achieves a 3.33% accuracy improvement on CIFAR-10 at IPC 1, highlighting its superior efficiency in extreme compression scenarios.

The remainder of this paper is structured as follows. Section 2 reviews related work of DD field. Section 3 details the CoDiM framework and its mathematical formulation. Section 4 presents the experimental setup and a quantitative evaluation of the results. Finally, Section 5 concludes the paper with a discussion on limitations and future research directions.

## II. RELATED WORK

Wang et al. [4] pioneered the field of DD, aimed at synthesizing compact, informative datasets from large-scale sources. They formulated the task as a meta-learning problem using a bi-level optimization framework, where the goal is to minimize the performance gap between models trained on distilled data versus original data. However, this approach requires unrolling the inner training loop, creating significant computational overhead and memory bottlenecks.

To reduce this cost, gradient matching methods such as DC [6] and DSA [7] match gradients from synthetic and real data, while distribution matching methods such as DM [8] and CAFE [9] align their feature distributions in a shared embedding space. To overcome the short-sightedness of one-step matching, MTT [10] aligns long-term training trajectories instead of only instantaneous gradients.

KIP [11] and FRePo [12] are categorized as kernel-based distillation methods. These approaches mitigate the computational overhead of the inner-loop training by employing Kernel Ridge Regression (KRR) with specific kernels such as Neural Tangent Kernel (NTK) [13]. Thereby effectively bypassing the need for costly iterative gradient descent in the inner loop. Recently, the field has witnessed a paradigm shift toward leveraging generative priors to DD. By integrating GANs, these novel approaches aim to synthesize more realistic samples while significantly reducing distillation time. For instance, DiM [14] employs a GAN-based framework coupled with a logit matching loss to generate high-quality data efficiently. Building on this concept, D2M [15] fine-tunes the generator using a combination of feature and logit matching losses. Furthermore, Li et al. [16] proposed a generative distillation method that standardizes logits post matching and incorporates a self-knowledge distillation loss to further improve performance. DAP [17] leverages a diffusion model and a Mercer kernel, formalizes the representativeness prior by quantifying feature space similarity, and introduces this prior as guidance to steer the reverse diffusion process.

## III. METHODOLOGY

### A. Problem Statement

The task of DD is to condense a large-scale dataset  $\mathcal{T} = \{(x_i, y_i)\}_{i=1}^{|\mathcal{T}|}$  into a distilled dataset  $\mathcal{S} = \{(\tilde{x}_j, \tilde{y}_j)\}_{j=1}^{|\mathcal{S}|}$  whose size is significantly smaller, while still preserving the critical information. The primary objective is to ensure that a model trained on  $\mathcal{S}$  can achieve a generalization performance comparable to a model trained on the full dataset  $\mathcal{T}$ . The distilled set  $\mathcal{S}$  can be formulated as the output of a mapping process  $f$  derived from  $\mathcal{T}$ , parameterized and optimized via a learning model. Formally,  $\mathcal{S}$  is modeled as:

$$\mathcal{S} = f(\mathcal{T}; \mathcal{S}_0; \theta) \quad (1)$$

where:

- $f$  denotes the mapping function (or generator) parameterized by  $\theta$ .
- $\mathcal{S}_0$  represents the initial state of the distilled dataset (e.g., random noise or randomly sampled images) at the beginning of the optimization loop.

Consequently, this process is defined as an optimization task where the goal is to extract a compact set  $\mathcal{S}$  from  $\mathcal{T}$  that approximates the training effectiveness of the full dataset. This approach facilitates significant reductions in computational, memory, and transmission costs without compromising the model's performance quality.

### B. Contrastive Distill into Model

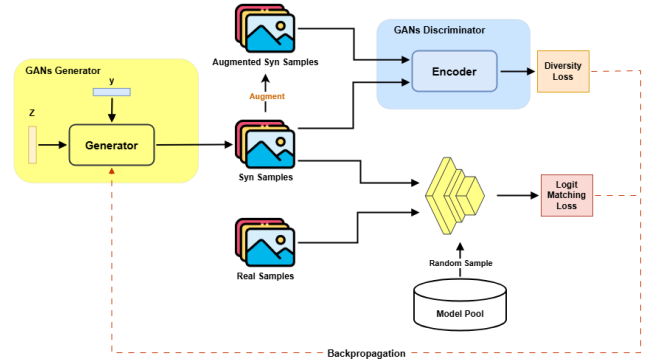


Fig. 1: CoDiM architecture utilizes two loss functions to facilitate high-quality synthetic data generation. InfoNCE loss promotes diversity by comparing representative vectors of synthetic samples and their augmented versions after passing through an encoder. Logit matching loss ensures that the images produced by the generator effectively inherit essential information and structural characteristics from the original dataset.

1) *Framework overview:* We propose CoDiM, a two-stage framework for dataset distillation. The pre-training stage employs an Auxiliary Classifier GAN (AC-GAN) to initialize the generator for class-conditional sample synthesis. In the fine-tuning stage, the generator is further optimized using a logit matching loss to align synthetic data with the target

model’s decision boundaries. Additionally, an InfoNCE loss is incorporated in both stages to encourage semantic diversity and discriminative representations, mitigating mode collapse and improving the generalization ability of the distilled dataset.

2) *GANs Architecture*: We adopt the architecture of AC-GAN [18]. AC-GAN integrates an auxiliary classification branch, requiring the Discriminator to output both a validity score and a class probability. We find this design particularly suited for DD, as it compels the Generator to synthesize samples that strictly adhere to specific semantic labels, thereby preventing class misalignment and ensuring that the compact dataset  $\mathcal{S}$  remains representative of the target distribution. The optimization objectives for the Discriminator ( $L_D$ ) and Generator ( $L_G$ ) are composed of the log-likelihood of the correct source and the correct class, defined as follows:

$$L_D = \mathbb{E}_{x, c \sim p_{\text{data}}} [-\log D_{\text{src}}(x) - \log D_{\text{cls}}(c | x)] \\ + \mathbb{E}_{z, c \sim p_{z, c}} [-\log(1 - D_{\text{src}}(G(z, c))) \\ - \log D_{\text{cls}}(c | G(z, c))] \quad (2)$$

$$L_G = \mathbb{E}_{z, c \sim p_{z, c}} [-\log D_{\text{src}}(G(z, c)) - \log D_{\text{cls}}(c | G(z, c))] \quad (3)$$

where  $D_{\text{src}}$  denotes the probability distribution over sources (real vs. fake), and  $D_{\text{cls}}$  represents the probability distribution over class labels given the input.

3) *Logit Matching Loss*: We adopt the Logit Matching strategy inspired by DiM [14]. The formulation of the logit loss is presented in Equation 4. This objective compels the Generator  $G$  to synthesize samples  $\tilde{x}$  that, when processed by the target network  $f$ , produce output logits closely aligning with those yielded by real samples. Crucially, this loss ensures that the synthetic data encodes the essential training signals inherent in the original distribution, specifically gradients, decision boundaries, and class statistics, rather than merely mimicking superficial visual appearances.

$$\mathcal{L}_{\text{logit}} = \frac{1}{B} \sum_{i=1}^B \|f(x_i) - f(G(z_i, y_i))\|_2^2 \quad (4)$$

4) *Diversity Loss*: We leverage the InfoNCE loss to explicitly enforce diversity within the Generator. Let  $\tilde{x}_i^c$  denote a synthesized sample belonging to class  $c$ . This sample is mapped to a latent representation  $z_i^c = \phi(\tilde{x}_i^c)$  via a feature encoder  $\phi(\cdot)$ . Simultaneously, a positive view  $\tilde{x}_i^{c,+} = \text{augment}(\tilde{x}_i^c)$  is generated using standard label-preserving transformations and subsequently encoded as  $z_i^{c,+} = \phi(\tilde{x}_i^{c,+})$ . In our framework, the pair  $(z_i^c, z_i^{c,+})$  serves as a positive pair, encouraging the generator to maintain semantic invariance against superficial perturbations. Conversely, the remaining samples within the same class, denoted as  $\{z_j^c\}_{j \neq i}$ , are treated as intra-class negatives. By pushing these representations apart, the objective forces the generator to produce distinct, non-redundant samples for the same class. Samples from different classes  $\{z_k\}_{y_k \neq c}$  act as inter-class negatives, enhancing the separability of class manifolds in the feature space.

$$\mathcal{L}_{\text{div}} = -\frac{1}{N_c} \sum_{i=1}^{N_c} \log \frac{e^{\text{sim}(z_i^c, z_i^{c,+})/\tau}}{\sum_{z_k \in \mathcal{D}_i} e^{\text{sim}(z_i^c, z_k)/\tau}} \quad (5)$$

where:

- $k$  denotes the total number of samples in a training batch.
- $\text{sim}(u, v) = \frac{u^\top v}{\|u\| \|v\|}$  computes the cosine similarity between vectors  $u$  and  $v$ .
- $\tau > 0$  represents the temperature hyperparameter that controls the sharpness of the output distribution. A smaller  $\tau$  increases the model’s sensitivity to hard negatives (sharpening the distribution), whereas a larger  $\tau$  results in a smoother distribution with reduced discrimination penalties.
- $N_c$  indicates the number of synthetic samples generated for class  $c$  in the current batch.

5) *Encoder*: Instead of introducing an external network, we repurpose the Discriminator ( $D$ ) to serve as the feature encoder. This choice offers dual benefits: it ensures computational efficiency by minimizing parameter overhead while simultaneously capitalizing on the robust semantic feature extraction capabilities inherent in the AC-GAN framework. We extract the high-level feature representations from the penultimate layer of the Discriminator. These features are then mapped through a non-linear projection head to reduce the computational overhead for using InfoNCE loss.

6) *Model Pool*: To ensure robustness, we establish a Model Pool comprising a predefined collection of diverse neural network architectures tailored to the target dataset. At each iteration of the distillation process, the algorithm randomly samples a network from this pool to serve as the reference model. The primary role of this sampled model is to provide logits for loss computation, thereby defining the optimization objective that guides the Generator’s parameter updates. This strategy enables CoDiM to access a broad spectrum of feature representation spaces, effectively mitigating the risk of the synthesized data overfitting to a single specific architecture. To evaluate the framework’s adaptability across varied architectural complexities while maintaining computational efficiency, our Model Pool incorporates three representative architectures: ResNet-10, ResNet-18, and CNN-4. The architecture of CNN-4 is further detailed in Fig. 2.

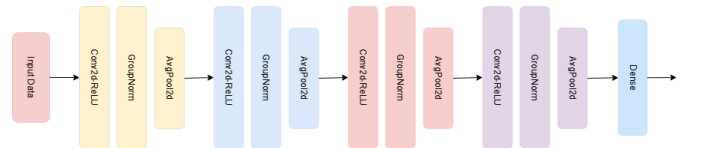


Fig. 2: CNN 4 architecture

7) *Optimization Process*: The CoDiM training protocol is divided into two phases to ensure stability and high fidelity generation. Stage 1 (Warm-up) focuses on establishing a robust feature space by integrating the diversity loss  $\mathcal{L}_{\text{div}}$  into the standard AC-GAN objective. Crucially, we apply  $\mathcal{L}_{\text{div}}$  asymmetrically: it is computed on real samples for the Discriminator ( $D$ ) to prevent feature clustering, and on synthetic samples for the Generator ( $G$ ) to explicitly mitigate

mode collapse. The objectives for this initialization stage are:

$$\begin{aligned}\mathcal{L}_D^{(1)} &= \mathcal{L}_D + \beta \mathcal{L}_{\text{div}}(x_{\text{real}}) \\ \mathcal{L}_G^{(1)} &= \mathcal{L}_G + \beta \mathcal{L}_{\text{div}}(x_{\text{fake}})\end{aligned}\quad (6)$$

Stage 2 (Distillation) activates the logit matching term  $\mathcal{L}_{\text{logit}}$  to distill knowledge from the surrogate model into the synthetic set  $\mathcal{S}$ . This guides the Generator to align with the target decision boundaries while maintaining the diversity constraints established in the warm-up phase. The complete training procedure (Algorithm 1) is governed by the following total loss functions, where  $\alpha, \beta$  balance the contribution of each component:

$$\begin{aligned}\mathcal{L}_D^{(2)} &= \mathcal{L}_D + \beta \mathcal{L}_{\text{div}}(x_{\text{real}}) \\ \mathcal{L}_G^{(2)} &= \mathcal{L}_G + \alpha \mathcal{L}_{\text{logit}} + \beta \mathcal{L}_{\text{div}}(x_{\text{fake}})\end{aligned}\quad (7)$$

---

**Algorithm 1** Contrastive Distill into Model (CoDiM)

---

**Require:** Generator  $G$ , Discriminator  $D$ , Model Pool  $\mathcal{M}$ , Pre-training epochs  $N$ , Matching epochs  $Q$ , Number of batches  $J$ , Temperature  $\tau$ , Learning Rate  $\eta$ , Hyperparameters  $\alpha, \beta$ .

**Ensure:** Optimized Generator parameters  $\theta_G^*$

```

1: // Stage 1: Warm-up
2: for  $t = 1$  to  $N$  do
3:   for  $j = 1$  to  $J$  do
4:     Sample real data batch  $x_{\text{real}}$  and latent noise  $z$ 
5:     Compute  $\mathcal{L}_D^{(1)}$  and  $\mathcal{L}_G^{(1)}$  via Eq. 6
6:     Update  $\theta_D \leftarrow \theta_D - \eta \nabla \mathcal{L}_D^{(1)}$ 
7:     Update  $\theta_G \leftarrow \theta_G - \eta \nabla \mathcal{L}_G^{(1)}$ 
8:   end for
9: end for
10: // Stage 2: Distillation
11: for  $k = 1$  to  $Q$  do
12:   for  $j = 1$  to  $J$  do
13:     Sample real data batch  $x_{\text{real}}$  and latent noise  $z$ 
14:     Sample a surrogate model  $f \sim \mathcal{M}$ 
15:     Compute  $\mathcal{L}_D^{(2)}$  and  $\mathcal{L}_G^{(2)}$  via Eq. 7
16:     Update  $\theta_D \leftarrow \theta_D - \eta \nabla \mathcal{L}_D^{(2)}$ 
17:     Update  $\theta_G \leftarrow \theta_G - \eta \nabla \mathcal{L}_G^{(2)}$ 
18:   end for
19: end for
20: return  $\theta_G$ 
```

---

The CoDiM optimization procedure is summarized in Algorithm 1. A detailed line-by-line breakdown is provided below:

- **Lines 2–3:** The algorithm initiates a nested loop structure, iterating through  $N$  pre training epochs.
- **Line 5:** The objective functions are calculated according to Eq. 6.
- **Lines 6–7:** The parameters of the Discriminator ( $\theta_D$ ) and Generator ( $\theta_G$ ) are updated via gradient descent using the learning rate  $\eta$ .
- **Lines 11–12:** Start distillation phase, which runs for  $Q$  matching epochs.

- **Line 14:** A surrogate model  $f$  is randomly sampled from the Model Pool  $\mathcal{M}$ .
- **Line 15:** The distillation objectives are computed based on Eq. 7.
- **Lines 16–17:** Parameters  $\theta_D$  and  $\theta_G$  are updated.

## IV. EVALUATION

### A. Experimental Setup

**Datasets:** We evaluate CoDiM on three standard benchmarks: Fashion-MNIST [19] ( $28 \times 28$  grayscale images of fashion items), CIFAR-10 [20] ( $32 \times 32$  color images of natural objects), and SVHN [21] ( $32 \times 32$  color images of house numbers). All datasets comprise 10 classes.

**Baselines:** We evaluate CoDiM against two categories of methods: (i) Coreset Selection, including Random selection, Herding, K-Center, and Forgetting; and (ii) Dataset Distillation (DD), specifically DC [6], DSA [7], DM [8], and the generative based DiM [14].

**Implementation Details:** All experiments are implemented in PyTorch and executed on a single NVIDIA GeForce RTX 4070 GPU. The training pipeline consists of a 120 epoch AC-GAN warm-up followed by a 100 epoch distillation stage, optimized using Adam ( $lr = 10^{-4}$ , batch size 128). We set the hyperparameters to  $\alpha = 0.01$ ,  $\beta = 0.1$ , and  $\tau = 0.1$ . To ensure deterministic reproducibility and maintain a consistent benchmark, we employ a fixed random seed of 42 for all evaluations, effectively isolating the algorithmic contributions of CoDiM from stochastic initialization variance.

**Evaluation Protocol:** We evaluate all methods by training a CNN-4 model (Fig. 2) for 300 epochs via SGD ( $lr = 0.01$ , momentum 0.9, weight decay  $5 \times 10^{-4}$ ). As traditional methods optimize for a single architecture, standardizing evaluation on CNN-4 ensures a fair comparison across all paradigms. Following DiM [14], we adopt the Image Number Per Class (INPC) metric to fully leverages the potential of GANs.

### B. Benchmark Comparison

In this subsection, we evaluate the performance of the proposed CoDiM against several DD baselines. Experiments are conducted with  $\text{IPC}/\text{INPC} \in \{1, 10, 50\}$ .

As shown in Table I, our method consistently outperforms traditional pixel-based methods. Notably, on more challenging datasets such as CIFAR-10 and SVHN, CoDiM demonstrates superior stability and accuracy, particularly in extremely small dataset scenarios ( $\text{IPC}/\text{INPC}=1$ ), where the generative capacity of our framework effectively captures the underlying distribution better than static pixel optimization.

The quantitative analysis also indicates a clear performance advantage for dataset distillation over coreset selection across all scenarios. At IPC 1, CoDiM achieves 72.72% on Fashion-MNIST, outperforming the best coreset method by over 12%. For the more challenging CIFAR-10 task, CoDiM delivers 38.49% accuracy at IPC 1, which significantly exceeds the 16.75% produced by Random selection. These numerical trends suggest that synthetic data generation is far more effective than original image selection for representing complex distributions in low-budget settings.

TABLE I: Accuracy (%) comparison of coreset selection and dataset distillation methods across different image budgets

| Dataset       | IPC/INPC | Coreset Selection |         |          |            | Dataset Distillation |       |       |              |              | Full  |
|---------------|----------|-------------------|---------|----------|------------|----------------------|-------|-------|--------------|--------------|-------|
|               |          | Random            | Herding | K-Center | Forgetting | DC                   | DSA   | DM    | DiM          | CoDiM (Ours) |       |
| Fashion-MNIST | 1        | 48.44             | 60.44   | 49.85    | 31.1       | 69.69                | 68.25 | 69.27 | 71.04        | <b>72.72</b> | 92.72 |
|               | 10       | 73.36             | 77.16   | 50.78    | 56.2       | 72.77                | 72.07 | 72.73 | <b>82.89</b> | 82.06        |       |
|               | 50       | 82.0              | 84.17   | 67.63    | 59.42      | 80.70                | 78.66 | 70.76 | <b>85.89</b> | 84.86        |       |
| SVHN          | 1        | 12.72             | 18.52   | 19.86    | 19.13      | 25.77                | 22.40 | 17.55 | 64.30        | <b>67.85</b> | 92.46 |
|               | 10       | 34.79             | 35.26   | 16.68    | 21.68      | 54.96                | 50.41 | 42.44 | 77.21        | <b>77.55</b> |       |
|               | 50       | 65.13             | 66.8    | 28.41    | 33.65      | 68.34                | 64.19 | 44.57 | 83.00        | <b>83.78</b> |       |
| CIFAR-10      | 1        | 16.75             | 20.7    | 15.46    | 16.02      | 27.80                | 26.69 | 26.33 | 35.16        | <b>38.49</b> | 83.17 |
|               | 10       | 28.54             | 35.13   | 14.67    | 24.2       | 36.21                | 35.57 | 32.14 | 53.46        | <b>57.1</b>  |       |
|               | 50       | 44.35             | 48.86   | 29.57    | 34.13      | 46.50                | 47.26 | 31.54 | 66.78        | <b>67.01</b> |       |

CoDiM demonstrates consistent scalability across varying image budgets. On CIFAR-10, accuracy improves from 38.49% to 67.01% as the budget grows from 1 to 50 IPC. A similar trend is observed in SVHN, where CoDiM maintains a lead over the DiM baseline, peaking at 83.78% at IPC 50. While DiM slightly outperforms our method on Fashion-MNIST at higher IPC values, CoDiM achieves higher accuracy in the majority of other evaluations. These results suggest that the proposed framework is particularly effective as data complexity increases, establishing CoDiM as a robust approach for high-quality dataset distillation.

To evaluate data efficiency, we report the recovery rate, defined as the ratio between the accuracy of the distilled set and the full dataset ( $\text{Recovery} = \frac{\text{Acc}_{\text{distilled}}}{\text{Acc}_{\text{full}}} \times 100\%$ ). As shown in Fig. 3, CoDiM recovers strong performance with very limited samples. On SVHN, using only 0.683% of the data ( $\text{IPC} = 50$ ), CoDiM achieves a recovery rate of 90.61%, surpassing DiM (89.76%). The advantage is more pronounced on CIFAR-10, where CoDiM recovers 68.65% at  $\text{IPC} = 10$ , compared to 64.28% for DiM. These results indicate that CoDiM achieves higher accuracy while utilizing the data budget more effectively, particularly under extreme compression.

### C. Ablation Study

In this subsection, we evaluate hyperparameter sensitivity and component contributions to optimize CoDiM for high-quality data generation, establishing a balanced configuration that justifies the necessity of our proposed distillation methodology across various scenarios.

TABLE II: Ablation study of  $\beta$ : Accuracy (%) on CIFAR-10

| INPC | $\beta$ value |       |       |
|------|---------------|-------|-------|
|      | 0.03          | 0.1   | 0.3   |
| 1    | 36.8          | 38.49 | 35.4  |
| 10   | 54.02         | 57.1  | 53.18 |
| 50   | 61.86         | 67.01 | 61.89 |

Initially, we investigate the sensitivity of the model to the  $\beta$  parameter while maintaining a fixed  $\alpha$  value of 0.01, which allows for a clear observation of how the weight of  $\beta$  influences the representative power of the synthetic images. Table II presents the accuracy on the CIFAR-10 dataset across different image budgets. The quantitative results demonstrate that the model reaches peak performance when  $\beta$  is set to 0.1 for all INPC budgets. At INPC 50, a  $\beta$  value of 0.1 achieves 67.01% accuracy, which is approximately 5% higher than the results obtained at 0.03 or 0.3. A similar pattern is observed at INPC

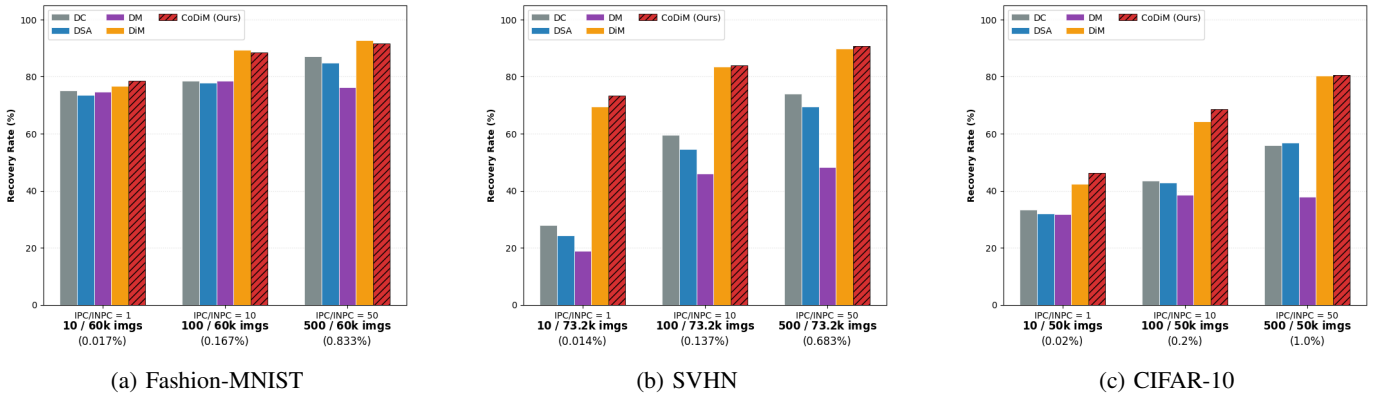


Fig. 3: Performance comparison of CoDiM against dataset distillation methods on Fashion-MNIST, SVHN, and CIFAR-10. Our method consistently achieves higher recovery rate across almost every case, significantly narrowing the gap to the full dataset performance.

10, where the accuracy of 57.1% remains superior to other configurations by at least 3%. These numerical improvements confirm that 0.1 is the ideal weight for the  $\beta$  parameter to ensure stable training and maximize classification precision.

In addition to the parameter search, we evaluate a baseline configuration where only the GAN architecture is utilized to demonstrate that the generative model alone is insufficient for high-quality dataset distillation. Table III provides a direct comparison between the standalone GAN results and the full CoDiM framework. The experimental results show a significant performance decrease across all categories when the specific enhancements of our method are absent. For the Fashion-MNIST dataset at INPC 10, the GAN baseline only reaches 77.76% which is 4.3% lower than CoDiM.

In the CIFAR-10 experiments at INPC 50, the accuracy drops from 67.01% to 59.16% representing a significant loss of 7.85%. These findings confirm that while the GAN component provides a useful foundation for image synthesis, the improvements introduced in our research are essential for reaching competitive precision. The larger margin of improvement observed in CIFAR-10 suggests that CoDiM offers greater advantages when dealing with datasets that possess higher diversity and complexity.

TABLE III: Accuracy (%) comparison between the GAN baseline and CoDiM

| INPC | Fashion-MNIST |              | CIFAR-10 |              |
|------|---------------|--------------|----------|--------------|
|      | GAN Only      | CoDiM (Ours) | GAN Only | CoDiM (Ours) |
| 1    | 71.79         | 72.72        | 34.75    | 38.49        |
| 10   | 77.76         | 82.06        | 50.13    | 57.10        |
| 50   | 82.24         | 84.86        | 59.16    | 67.01        |

**Discussion and Future Work:** While CoDiM establishes a robust generative distillation paradigm, future research will focus on: (i) expanding the model pool with diverse architectures to assess cross architecture generalization; (ii) transitioning from GANs to other generative models to overcome resolution constraints and enhance distribution diversity; and (iii) scaling the framework to higher resolution benchmarks like ImageNet-1K to validate its effectiveness in large scale scenarios.

## V. CONCLUSION

This paper presented CoDiM, a novel generative dataset distillation framework that bridges the gap between GAN-based synthesis and Contrastive Learning. By shifting from traditional pixel-space optimization to parameterized representations, CoDiM effectively captures the underlying source data distribution for efficient model training. Our empirical evaluations demonstrate that CoDiM consistently outperforms state-of-the-art methods, notably achieving accuracy gains of 3.55% on SVHN and 3.33% on CIFAR-10 at IPC 1. These results validate the effectiveness and scalability of our approach in generating high-quality, representative synthetic datasets across various image budgets, providing a robust solution for large-scale dataset condensation.

## REFERENCES

- [1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [2] H. Amer, A. H. Salamah, A. Sajedi, and E.-H. Yang, “High performance convolution using sparsity and patterns for inference in deep convolutional neural networks,” *arXiv preprint arXiv:2104.08314*, 2021.
- [3] N. C. Thompson, K. Greenewald, K. Lee, and G. F. Manso, “Deep learning’s diminishing returns: The cost of improvement is becoming unsustainable,” *IEEE Spectrum*, vol. 58, no. 10, pp. 50–55, 2021.
- [4] T. Wang, J.-Y. Zhu, A. Torralba, and A. A. Efros, “Dataset distillation,” *arXiv preprint arXiv:1811.10959*, 2018.
- [5] S. Lei and D. Tao, “A comprehensive survey of dataset distillation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 1, pp. 17–32, 2023.
- [6] B. Zhao, K. R. Mopuri, and H. Bilen, “Dataset condensation with gradient matching,” *arXiv preprint arXiv:2006.05929*, 2020.
- [7] B. Zhao and H. Bilen, “Dataset condensation with differentiable siamese augmentation,” in *International Conference on Machine Learning*. PMLR, 2021, pp. 12674–12685.
- [8] —, “Dataset condensation with distribution matching,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2023, pp. 6514–6523.
- [9] K. Wang, B. Zhao, X. Peng, Z. Zhu, S. Yang, S. Wang, G. Huang, H. Bilen, X. Wang, and Y. You, “Cafe: Learning to condense dataset by aligning features,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 196–12 205.
- [10] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, “Dataset distillation by matching training trajectories,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 4750–4759.
- [11] T. Nguyen, R. Novak, L. Xiao, and J. Lee, “Dataset distillation with infinitely wide convolutional networks,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 5186–5198, 2021.
- [12] Y. Zhou, E. Nezhadarya, and J. Ba, “Dataset distillation using neural feature regression,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 9813–9827, 2022.
- [13] A. Jacot, F. Gabriel, and C. Hongler, “Neural tangent kernel: Convergence and generalization in neural networks,” *Advances in neural information processing systems*, vol. 31, 2018.
- [14] K. Wang, J. Gu, J. Gu, H. Zhang, D. Zhou, Z. Zhu, W. Jiang, and Y. You, “Dim: Distilling dataset into generative model,” in *European Conference on Computer Vision*. Springer, 2024, pp. 42–59.
- [15] A. Sajedi, S. Khaki, L. Z. Liu, E. Amjadi, Y. A. Lawryshyn, and K. N. Plataniotis, “Data-to-model distillation: Data-efficient learning framework,” in *European Conference on Computer Vision*. Springer, 2024, pp. 438–457.
- [16] L. Li, G. Li, R. Togo, K. Maeda, T. Ogawa, and M. Haseyama, “Generative dataset distillation based on self-knowledge distillation,” in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5.
- [17] D. Su, H. Wu, H. Chen, Y. Shi, Y. Wang, X. Ye, and J. Zhu, “Diffusion models as dataset distillation priors,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026.
- [18] A. Odena, C. Olah, and J. Shlens, “Conditional image synthesis with auxiliary classifier gans,” in *International conference on machine learning*. PMLR, 2017, pp. 2642–2651.
- [19] H. Xiao, K. Rasul, and R. Vollgraf, “Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms,” *arXiv preprint arXiv:1708.07747*, 2017.
- [20] A. Krizhevsky, G. Hinton *et al.*, “Learning multiple layers of features from tiny images,” 2009.
- [21] P. Sermanet, S. Chintala, and Y. LeCun, “Convolutional neural networks applied to house numbers digit classification,” in *Proceedings of the 21st international conference on pattern recognition (ICPR2012)*. IEEE, 2012, pp. 3288–3291.